COMP 512 Project 2 – Performance Analysis of the Paxos Total-Order Layer

Group 19

1 Test Setup and Methodology

Implementation under test. Every move is ordered by its own Paxos instance (*slot*): the proposer sends PREPARE to all n processes, waits for a majority of promises, sends ACCEPT, waits for a majority of acceptances, and broadcasts DECIDE. There is no stable leader; each process proposes its own moves for the lowest slot it has not seen decided. A ballot is (*counter*, *process id*), where a new counter is the highest seen plus one and ties are ordered by process id. A refused proposer backs off for a random time in $[1, 5 \cdot 2^k]$ ms (k = consecutive failures, capped at 6) and retries. `broadcastTOMsg` blocks until a majority accepted, so each process has at most one outstanding move.

Environment and experiments. We used the provided `TreasureIslandAppAuto`, started by a parameterised copy of `run_tiapp_auto.sh`, with all n processes on one host (Apple M3, 8 cores, 16 GB, macOS, OpenJDK 21; GCL over loopback TCP; default `FINE` logging; sleep disabled with `caffeinate`). (A) $n \in \{2,3,5,7,9\}$ × interval $\in \{0,5,10,20,50,100,200\}$ ms, 150 moves per player, 2 repetitions with different seeds (70 runs). (B) Fairness: $n \in \{5,9\}$, interval 0, 200 moves per player, 5 repetitions (10 runs). In all 80 runs all island logs were identical and contained every move of every player.

Metrics. *Latency* is the time from the call to `broadcastTOMsg` until a majority has accepted the move, taken from our log timestamps; the first 5 moves of each player (connection set-up, JIT warm-up) are excluded. *System throughput* is the number of decided slots per second at player 1 between the 10th and 90th percentile of decision times, which excludes start-up and the tail where some players have finished. The *per-process acceptance rate* is a player's number of moves divided by the time from its first broadcast to its last acceptance. The provided move generator sleeps until *interval* ms have passed since its last move, but `Thread.sleep` overshoots by ≈2 ms on our machine (median gap 12.2 ms at a 10 ms setting, while Paxos took ≈1 ms). Each process therefore offers at most $1000/(\text{interval}+2)$ moves/s.

2 Impact of the Number of Players and the Interval

Latency (Fig. 1). With large intervals (100–200 ms) moves of different players rarely overlap and latency is 4–9 ms for every n: the cost of one uncontested instance, i.e. two round trips to a majority over loopback plus synchronous logging of every GCL message. As the interval shrinks, latency grows, and faster the more players there are: at 20 ms it is 2.6 ms for $n = 2$ but 32 ms for $n = 9$, and at interval 0 it reaches 41 ms (p95 200 ms) for $n = 9$. The cause is contention: several players propose for the same slot, only one wins, and the others are refused, back off and retry. Fig. 3(b) shows this directly: the number of rounds per committed move rises from 1.1–1.5 at 200 ms to 4.6 for $n = 9$ at interval 0, and every lost round adds a round trip plus a random backoff. Each round also costs $\approx 5n$ messages and all n JVMs share 8 cores, so rounds themselves get slower as n grows. For $n = 2$, latency at 50–200 ms (4.2–5.1 ms) is higher than at 10–20 ms (2.5 ms).

We attribute this to the processes idling between moves, so threads must be woken and the CPU leaves its low-power state for each move.

Throughput (Fig. 2). For large intervals the system is not the bottleneck: throughput equals the offered load $n \cdot 1000/(\text{interval}+2)$ (e.g. 43.7 moves/s for $n = 9$ at 200 ms, 4.9 per player), so it grows linearly with n and inversely with the interval. As the interval decreases, the curves stop following the offered load and flatten. This happens earlier for more players, because their offered load is higher and their capacity, limited by contention, is lower: $n = 9$ reaches its plateau of ≈180–195 moves/s already at 20 ms, $n = 7$ stays at 210–250 moves/s for 5–20 ms, while $n = 2$ follows the offered load down to 5 ms.

3 Fairness Between Processes

At large intervals all processes have the same acceptance rate (4.8–4.9 moves/s each at 200 ms), because every move is accepted long before the next one is generated. As the interval approaches 0 this changes (Fig. 4): the acceptance rate shows a clear upward trend with the process number in every repetition. With 5 players, process 5 is accepted 34% more often than process 1 (97.4 vs. 72.7 moves/s; mean latency 7.9 vs. 10.3 ms), and process 1 was the slowest in all 5 repetitions. With 9 players, processes 8 and 9 are ≈23% faster than process 1 (25.0 vs. 20.4 moves/s), and one of them was the fastest process in every repetition.

The responsible aspect of our algorithm is the ballot tie-break. Every proposer picks "highest counter seen + 1", so proposers starting at about the same time usually choose the *same* counter, and the tie is decided by the process id. Our ids are `host:port` with ports increasing with the player number, so a higher player always wins a tie. Table 1 confirms this from the acceptor logs of a separate 5-player run: process 5 received no tie refusals while process 1 received 382, and process 1 lost 3.8 times as many phase-1 rounds as process 5. Each lost tie costs the loser a refusal, a backoff and a retry. The randomised backoff and the adoption of the highest counter seen keep the bias bounded: no process starves. It could be removed by breaking ties with a random per-ballot number or rotating the tie-break priority with the slot number.

4 Maximum Throughput

Fig. 3(a) shows the throughput at interval 0, where every player re-proposes as soon as its previous move is accepted. The maximum is ≈585 moves/s with 2 players, ≈490–510 with 3 and 5 players (large variance for $n = 5$: 357–610 over 7 runs), and drops to ≈287 with 7 and ≈206 with 9 players. The rate does not need to reach interval 0 to flatten out: for $n = 9$, 5–20 ms already yield 180–195 moves/s, the same as interval 0, and for $n = 7$ the plateau starts at 10–20 ms. Only for $n = 2, 3$ does the curve keep rising down to 0, because even at 5 ms each player still sleeps ≈7 ms after each fast move, so throughput is limited by the offered load, not by Paxos. The maximum falls as players are added because (i) every instance needs $\approx 5n$ messages, (ii) contention grows (4.6 rounds per move at $n = 9$ vs. 1.3 at $n = 2$, each lost round adding a backoff), and (iii) all players share one machine (8 cores, n JVMs, synchronous logging). A stable leader that pipelines moves would avoid most collisions, but the assignment requires each process to propose its own moves.

Limitations. All numbers come from a single host over loopback; on the SOCS machines network round trips would add to every latency and change the share of time lost to contention.

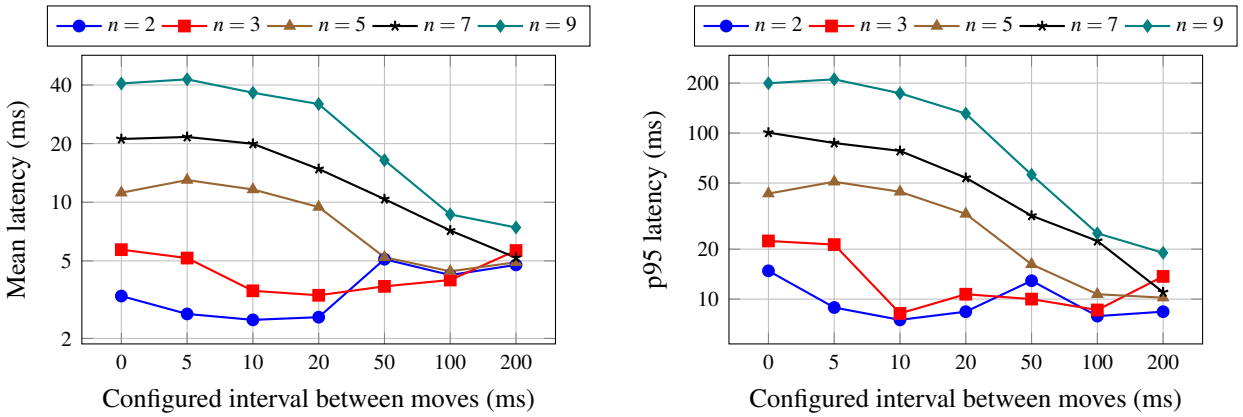

Figure 1: Time until a move is accepted (from `broadcastTOMsg` until a majority accepted it) vs. the configured interval, for n players. Left: mean; right: 95th percentile (both log scale; mean of 2 repetitions, 150 moves per player).

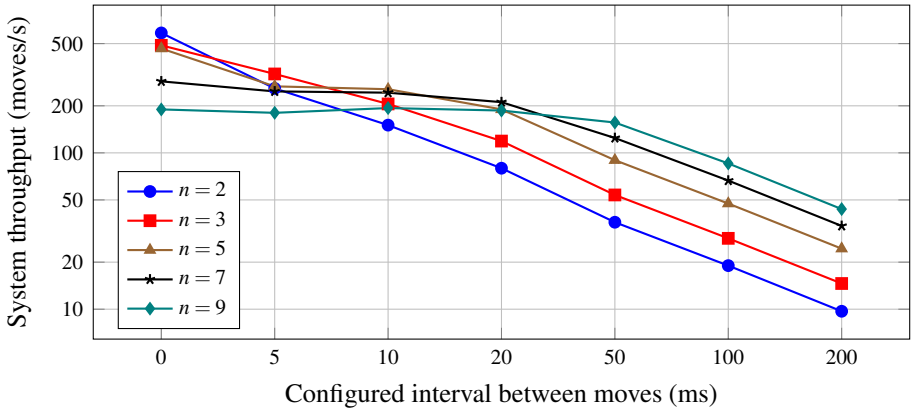

Figure 2: Moves decided per second by the whole system vs. the configured interval (log scale; mean of 2 repetitions). At large intervals the throughput equals the offered load $n \cdot 1000/(\text{interval} + 2)$; at small intervals it flattens at the capacity of the system, which decreases as n grows.

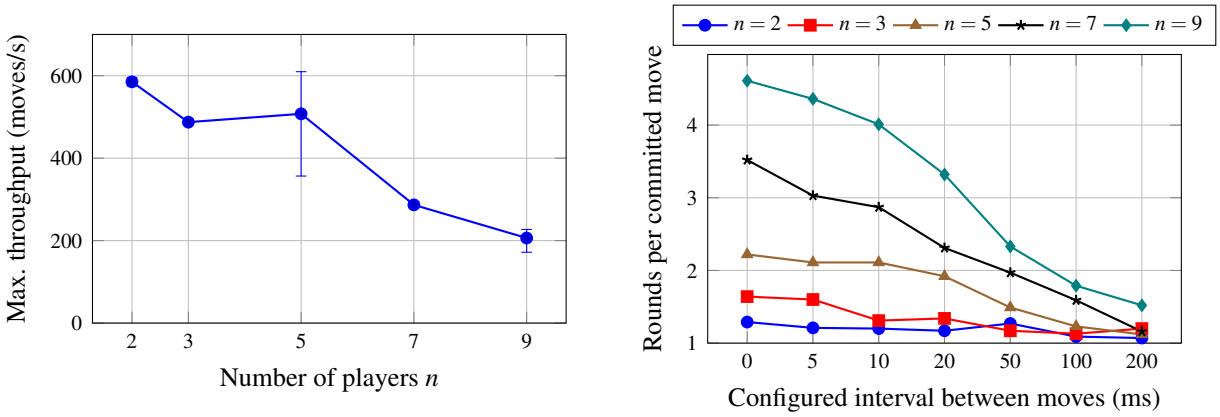

Figure 3: (a) Left: maximum throughput (interval 0) vs. number of players; mean with min/max over all runs (2 runs for $n = 2,3,7$; 7 runs for $n = 5,9$). (b) Right: contention, measured as the number of Paxos rounds started per committed move. 1.0 means every move succeeded in its first round.

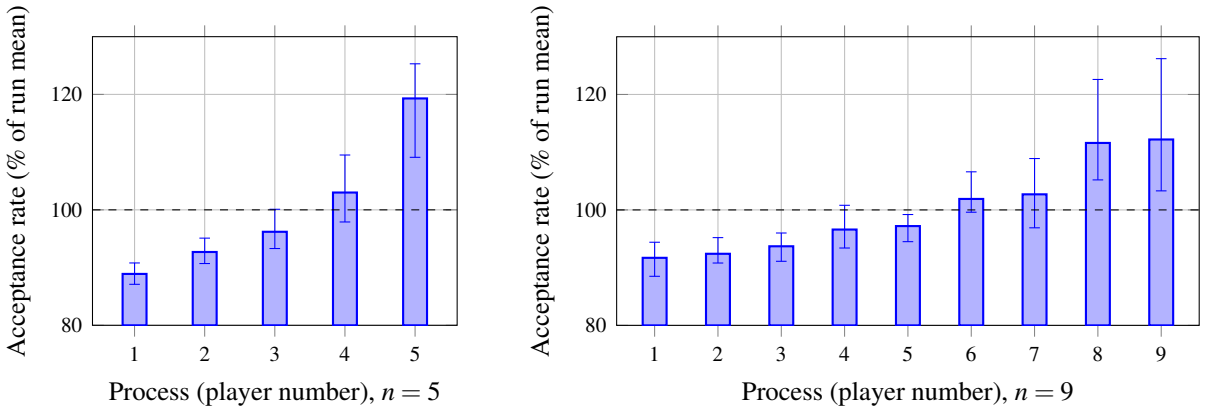

Figure 4: Per-process acceptance rate at interval 0, relative to the mean of all processes in the same run (100% = perfectly fair). Bars: mean of 5 repetitions; whiskers: min/max. Absolute means: 72.7–97.4 moves/s ($n = 5$) and 20.4–25.0 moves/s ($n = 9$).

Process (player)	1	2	3	4	5
Refusals because of an equal-counter tie lost to a higher id	382	301	320	187	0
Refusals because of a higher counter	188	263	330	325	328
Rounds lost in phase 1	69	53	61	34	18
Rounds lost in phase 2	61	68	76	78	59

Table 1: Why the higher-numbered processes are faster: the causes of refusals received by each proposer, counted from all acceptors' logs of one 5-player run at interval 0 (200 moves per player). Process 5 has the highest id and never loses a tie.